

调度

1. 工具

大数据常用的调度工具有：Oozie和Azkaban。

1.1 Oozie和Azkaban对比

1.Oozie是基于Hadoop系统进行操作，而Azkaban是基于命令行进行操作。使用hadoop提供的第三方包JobClient比直接在底层跑shell命令**开发成本小**，可能**遇到的坑也少**（一个是基于Hadoop平台，一个是基于Linux系统）。

2.Oozie的操作是放在Hadoop中，而Azkaban的运行是服务器运行shell命令。为**保证服务器的稳定**，使用Oozie靠谱点。

3.Oozie提供查询**任务执行状态**，Azkaban查询的是进程执行的结果，如果某进程执行的shell命令出错，其进程仍展示位成功，混淆了任务输出。

4.Oozie将任务执行的状态**持久化到数据库中**，Azkaban将任务的状态存储在服务器内存中，如果掉电，则Azkaban会丢失任务信息。

5.Oozie中定义的action类型更为丰富，而Azkaban中的依赖较为简单，当面对**复杂的逻辑**时**Oozie执行的比较顺畅**。

6.两者均可以通过WEB界面进行配置操作，Azkaban和Oozie均拥有自带WEB，而**Oozie**同时还**被HUE集成**，通过HUE集成管理，可以更加方便的在Web页面上完成workflow的启动、停止、恢复。

7.**CDH原生支持Oozie**，而Azkaban还需要额外的编译及安装配置。

1.2 项目选型

基于以上对比，本课程采用被ClouderaManager和Hue所支持的Oozie来进行调度。

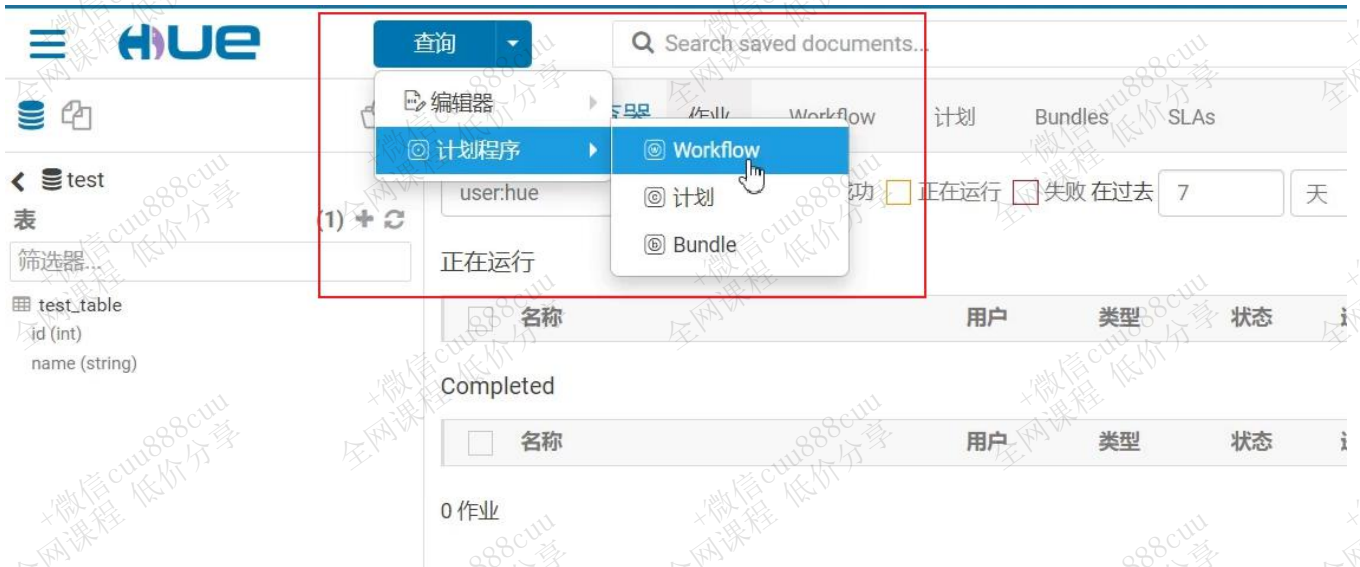


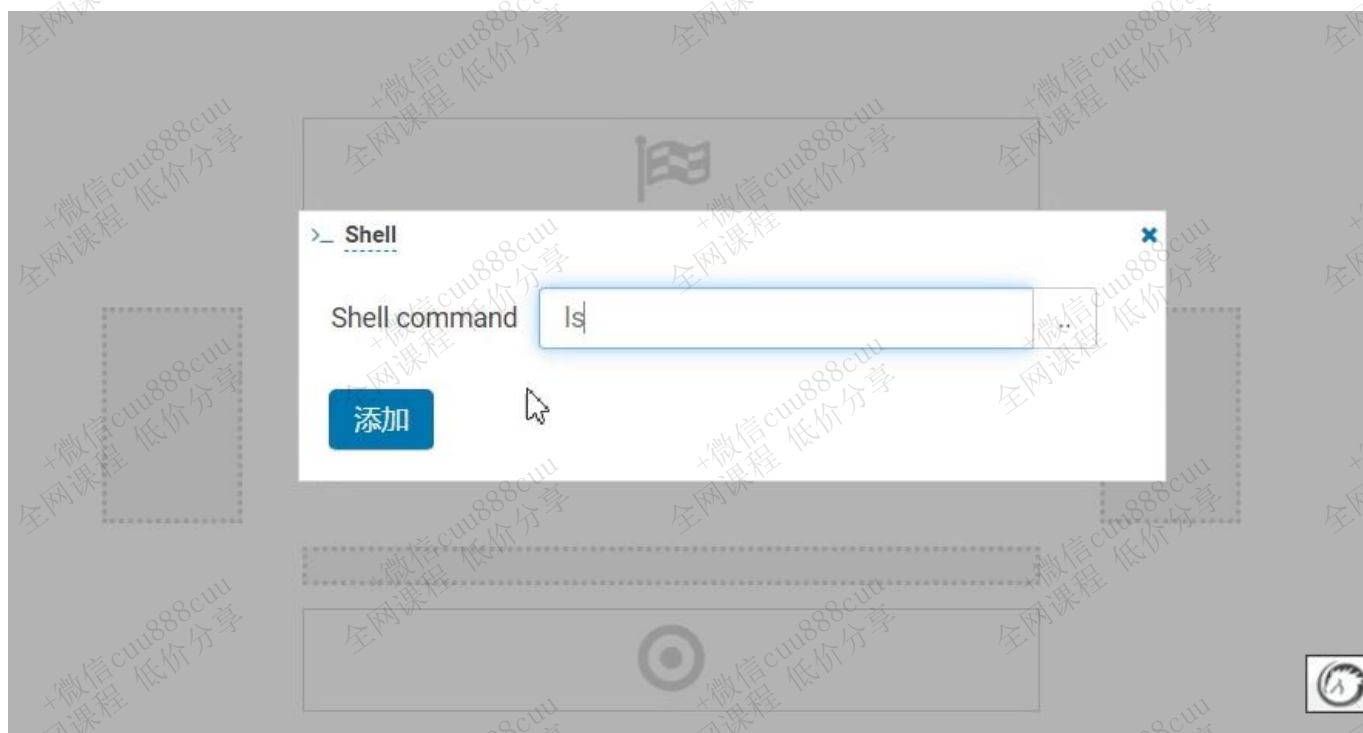
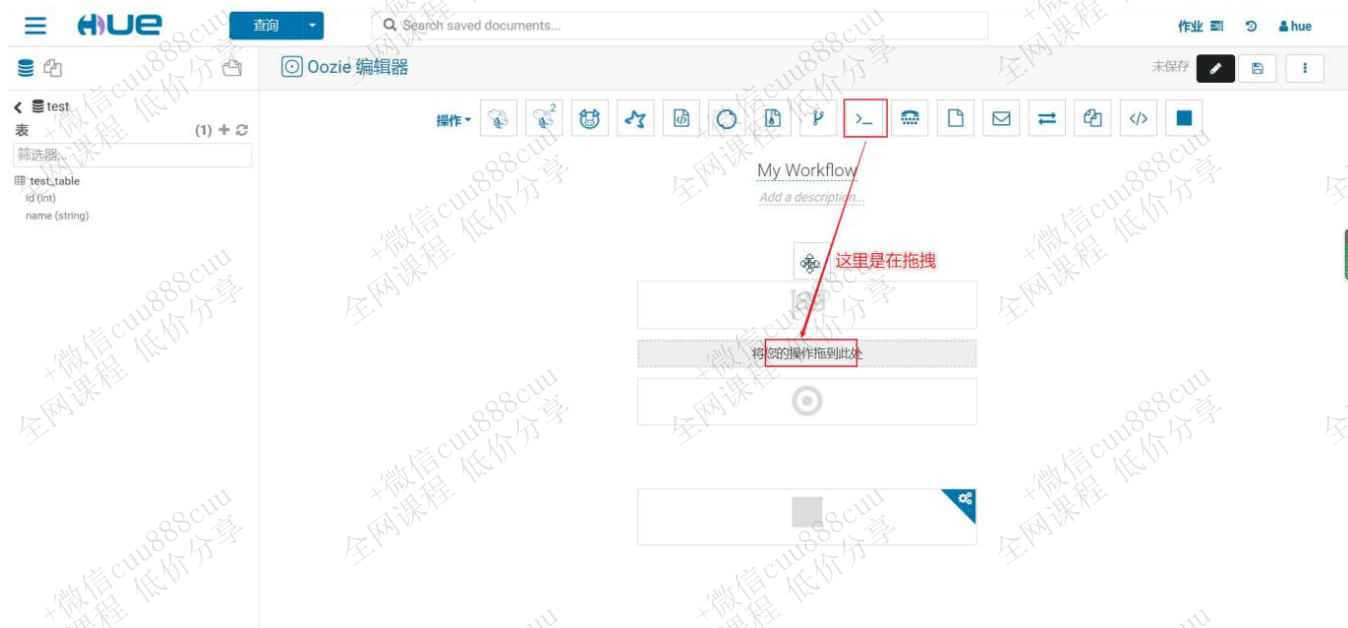
1.3 Oozie使用案例

1.3.1 查看已有的oozie程序



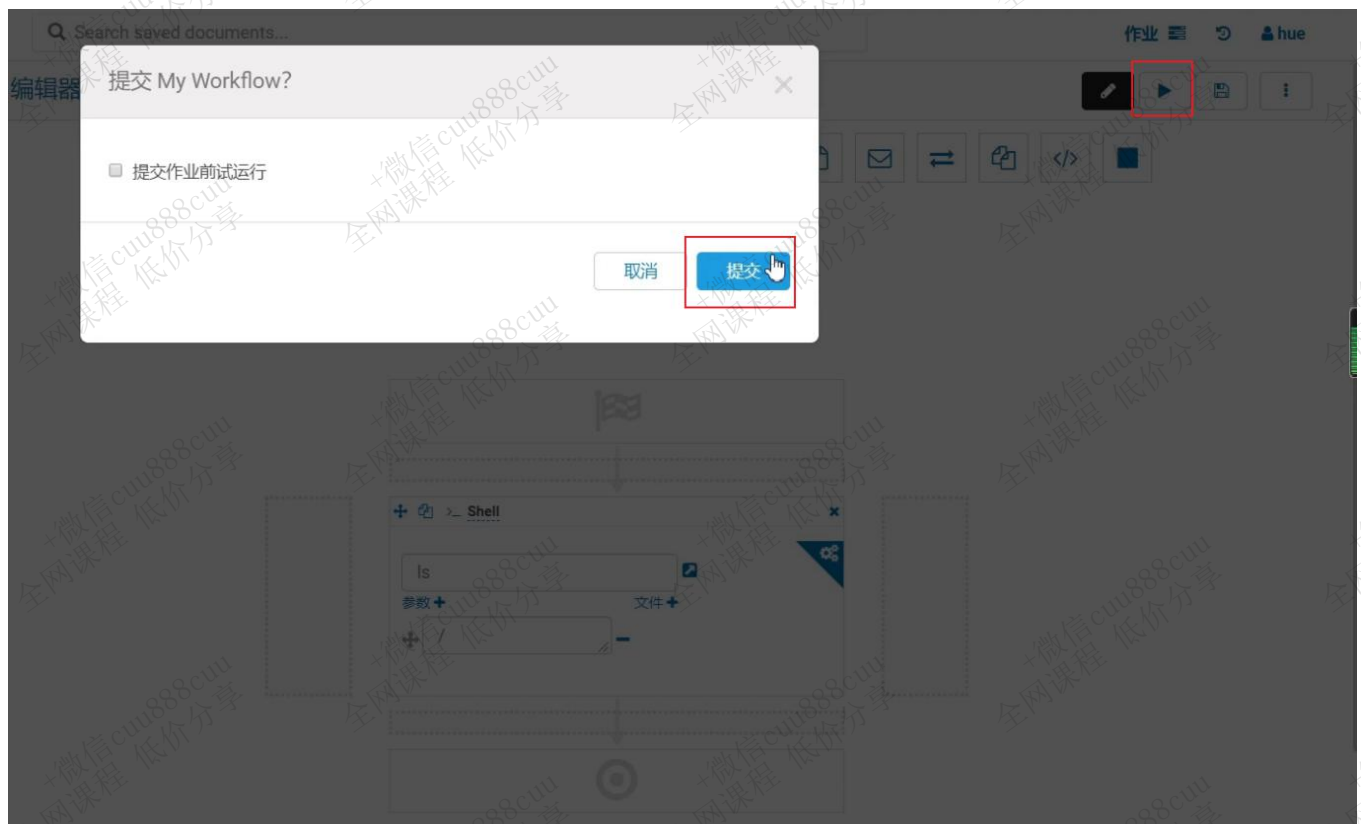
1.3.2 新增oozie程序







1.3.3 提交任务



作业浏览器

作业

Workflow

计划

Bundles

SLAs

My Workflow

配置

ID

0000000-200320165009

278-oozie-oozi-W

文档

My Workflow i

状态

RUNNING

用户

hue

进度

50%

持续时间

0s

已提交

2020年3月20日下午5点...

图

属性

日志

任务

XML

恢复

暂停

重新运行

停止

```

at org.apache.hadoop.ipc.Client.call(Client.java:1355)
at org.apache.hadoop.ipc.ProtobufRpcEngine$Invoker.invoke(ProtobufRpcEngine.java:228)
at org.apache.hadoop.ipc.ProtobufRpcEngine$Invoker.invoke(ProtobufRpcEngine.java:116)
at com.sun.proxy.$Proxy37.submitApplication(Unknown Source)
at org.apache.hadoop.yarn.api.impl.pb.client.ApplicationClientProtocolPBClientImpl.submitApplication(ApplicationClientProtocolPBClientImpl.java:281)
at sun.reflect.NativeMethodAccessorImpl.invoke0(Native Method)
at sun.reflect.NativeMethodAccessorImpl.invoke(NativeMethodAccessorImpl.java:62)
at sun.reflect.DelegatingMethodAccessorImpl.invoke(DelegatingMethodAccessorImpl.java:43)
at java.lang.reflect.Method.invoke(Method.java:498)
at org.apache.hadoop.io.retry.RetryInvocationHandler.invokeMethod(RetryInvocationHandler.java:422)
at org.apache.hadoop.io.retry.RetryInvocationHandler$Call.invokeMethod(RetryInvocationHandler.java:165)
at org.apache.hadoop.io.retry.RetryInvocationHandler$Call.invoke(RetryInvocationHandler.java:157)
at org.apache.hadoop.io.retry.RetryInvocationHandler$Call.invokeOnce(RetryInvocationHandler.java:95)
at org.apache.hadoop.io.retry.RetryInvocationHandler.invoke(RetryInvocationHandler.java:359)
at com.sun.proxy.$Proxy38.submitApplication(Unknown Source)
at org.apache.hadoop.yarn.client.api.impl.YarnClientImpl.submitApplication(YarnClientImpl.java:290)
at org.apache.oozie.action.hadoop.JavaActionExecutor.submitLauncher(JavaActionExecutor.java:1094)
... 11 more
2020-03-20 17:38:07,669 INFO org.apache.oozie.command.wf.ActionStartXCommand: SERVER[hadoop01] USER[hue] GROUP[-] TOKEN[] APP[My Workflow] JOB[0000000-200320165009278-oozie-oozi-W] ACTION[0000000-200320165009278-oozie-oozi-W@shell-7bd1] Next Retry, Attempt Number [1] in [10,000] milliseconds

```

1.3.4 修复异常

org.apache.oozie.action.ActionExecuterException: JA009: Invalid resource request! Cannot allocate containers as requested resource is greater than maximum allowed allocation.

Requested resource type=[memory-mb], Requested resource=<memory:2048, vCores:1>, maximum allowed allocation=<memory:1024, vCores:4>, please note that maximum allowed allocation is calculated by scheduler based on maximum resource of registered NodeManagers, which might be less than configured maximum allocation=<memory:1024, vCores:4>

这是内存分配导致的错误，需要设置内存并重启Yarn：将maximum改为和nodemanager内存一样的值。

CDH Test1

YARN (MR2 Included)

操作

状态

实例

配置

命令

应用程序

资源池

图表库

审核

Web UI

快速链接

yarn.scheduler.maximum-allocation-mb

筛选器

范围

YARN (MR2 Included) (服务范围... 0)

最大容器内存

yarn.scheduler.maximum-allocation-mb

ResourceManager Default Group

5632

兆字节

5.5 吉字节



CDH Test1

YARN (MR2 Included)

操作

状态 实例 配置 命令 应用程序 资源池 图表库 审核 Web UI 快速链接

yarn.nodemanager.resource.memory-mb

筛选器

范围

YARN (MR2 Included) (服务范... 0
Gateway 0

容器内存

yarn.nodemanager.resource.memory-mb
编辑单个值

NodeManager Default Group ...and 1 other

5632

兆字节

5.5 吉字节

1.3.5 修改后重启服务

Cloudera Manager

集群 主机 诊断 审核 图表 管理

搜索 支持 admin

CDH Test1

过期配置

筛选器

清除全部

文件

文件: hive-conf/yarn-site.xml 0
文件: yarn-conf/yarn-site.xml 1
文件: yarn-site.xml 3

服务

清除

Hive 2
Hue 2
Oozie 2
Spark 1
YARN (MR2 Included) 4

角色类型

Hive Metastore Server 1
HiveServer2 1
Hue Server 1
JobHistory Server 1
NodeManager 1
Oozie Server 1
ResourceManager 1

文件: yarn-conf/yarn-site.xml

5 个服务受影响 显示

```
... 47 47 <value>1</value>
48 48 </property>
49 49 <property>
50 50 <name>yarn.scheduler.minimum-allocation-mb</name>
51 51 <value>1024</value>
52 52 </property>
53 53 <property>
54 54 <name>yarn.scheduler.increment-allocation-mb</name>
55 55 <value>512</value>
```

文件: yarn-site.xml

yarn(1) 显示

```
... 55 55 <value>1</value>
56 56 </property>
57 57 <property>
58 58 <name>yarn.scheduler.minimum-allocation-mb</name>
59 59 <value>1024</value>
60 60 </property>
61 61 <property>
62 62 <name>yarn.scheduler.increment-allocation-mb</name>
63 63 <value>512</value>
```

文件: yarn-site.xml

yarn(2) 显示

```
... 67 67 <value>1</value>
68 68 </property>
```

重启过时服务

1. 执行成功



作业浏览器 作业 Workflow 计划 Bundles SLAs

My Workflow

ID: 0000000-200320174311038-oozie-oozi-W

图 属性 日志 任务 XML

恢复 暂停 重新运行

文档: My Workflow

状态: SUCCEEDED

用户: hue

进度: 100%

持续时间: 4m, 31s

已提交: 2020年3月20日下午...

```
0174311038-oozie-oozi-W@End with user-retry state : userRetryCount [0], userRetryMax [0], userRetryInterval [10]
2020-03-20 17:51:54,321 INFO org.apache.oozie.action.control.EndActionExecutor: SERVER[hadoop01] USER[hue] GROUP[-] TOKEN[] APP
[My Workflow] JOB[0000000-200320174311038-oozie-oozi-W] ACTION[0000000-200320174311038-oozie-oozi-W@End] Starting action
2020-03-20 17:51:54,327 INFO org.apache.oozie.command.wf.ActionStartXCommand: SERVER[hadoop01] USER[hue] GROUP[-] TOKEN[] APP[My
Workflow] JOB[0000000-200320174311038-oozie-oozi-W] ACTION[0000000-200320174311038-oozie-oozi-W@End] [***0000000-200320174311038
-oozie-oozi-W@End***]Action status=DONE
2020-03-20 17:51:54,327 INFO org.apache.oozie.command.wf.ActionStartXCommand: SERVER[hadoop01] USER[hue] GROUP[-] TOKEN[] APP[My
Workflow] JOB[0000000-200320174311038-oozie-oozi-W] ACTION[0000000-200320174311038-oozie-oozi-W@End] [***0000000-200320174311038
-oozie-oozi-W@End***]Action updated in DB!
2020-03-20 17:51:54,347 INFO org.apache.oozie.action.control.EndActionExecutor: SERVER[hadoop01] USER[hue] GROUP[-] TOKEN[] APP
[My Workflow] JOB[0000000-200320174311038-oozie-oozi-W] ACTION[0000000-200320174311038-oozie-oozi-W@End] Action ended with exte
nal status [OK]
2020-03-20 17:51:54,395 INFO org.apache.oozie.command.wf.WorkflowNotificationXCommand: SERVER[hadoop01] USER[-] GROUP[-] TOKEN
[-] APP[-] JOB[0000000-200320174311038-oozie-oozi-W] ACTION[0000000-200320174311038-oozie-oozi-W@End] No Notification URL is def
ined. Therefore nothing to notify for job 0000000-200320174311038-oozie-oozi-W
2020-03-20 17:51:54,395 INFO org.apache.oozie.command.wf.WorkflowNotificationXCommand: SERVER[hadoop01] USER[-] GROUP[-] TOKEN
[-] APP[-] JOB[0000000-200320174311038-oozie-oozi-W] ACTION[0000000-200320174311038-oozie-oozi-W@shell-588f] No Notification URL
is defined. Therefore nothing to notify for job 0000000-200320174311038-oozie-oozi-W@shell-588f
2020-03-20 17:51:54,420 INFO org.apache.oozie.command.wf.WorkflowNotificationXCommand: SERVER[hadoop01] USER[-] GROUP[-] TOKEN
[-] APP[-] JOB[0000000-200320174311038-oozie-oozi-W] ACTION[] No Notification URL is defined. Therefore nothing to notify for jo
b 0000000-200320174311038-oozie-oozi-W
```

2. shell脚本

2.1 date命令

2.1.1 获取今天的日期

```
date
date +%Y%m%d
```

2.1.2 指定日期获取内容 -d或--date=

```
#获取指定日期的年月日格式输出
date -d "2014-11-12" +%Y%m%d

#获取指定日期的星期（周几）格式输出
```





```
date --date="2014-11-23" +%w
```

2.1.3 日期加减 -d或--date==

#获取上周日期 (day,month,year,hour)

```
date -d "-1 week" +%Y%m%d
```

#获取昨天日期

```
date -d '-1 day' +%Y-%m-%d
```

```
date --date="-24 hour" +%Y%m%d
```

日期格式化规则：

%a 当前域的星期缩写 (Sun..Sat)

%A 当前域的星期全写 (Sunday..Saturday)

%b 当前域的月份缩写 (Jan..Dec)

%B 当前域的月份全称 (January..December)

%d 两位的天 (01..31)

%D 短时间格式 (mm/dd/yy)

%e 短格式天 (1..31)

%F 文件时间格式 same as %Y-%m-%d

%h same as %b

%H 24小时制的小时 (00..23)

%I 12小时制的小时 (01..12)

%j 一年中的第几天 (001..366)

%k 短格式24小时制的小时 (0..23)

%m 双位月份 (01..12)

%M 双位分钟 (00..59)

%r 12小时制的时间表示 (时:分:秒,双位) time, 12-hour (hh:mm:ss [AP])M

%R 24小时制的时间表示 (时:分,双位) time, 24-hour (hh:mm)





%s 自基础时间 1970-01-01 00:00:00 到当前时刻的秒数(a GNU extension)

%T 24小时制时间表示(hh:mm:ss)

%u 数字表示的星期 (从星期一开始 1-7)

%x 本地日期格式 (mm/dd/yy)

%X 本地时间格式 (%H:%M:%S)

%y 两位的年(00..99)

%Y 年 (1970...)

2.2 \$() 与 \${}、`` 与 ()、\$(())

2.2.1 变量替换

在 bash shell 中，**\$()**是用来作变量替换的。

一般情况下，\$var与\${var}是没有区别的，但是用\${}会比较精确的界定变量名称的范围。

```
A=Linux
```

```
echo $AB      #表示变量AB
```

```
echo ${A}B    #表示变量A后连接着B
```

2.2.2 命令替换

在 bash shell 中，**\$()**与**` `**（反引号）都是用来作命令替换的。

命令替换与变量替换差不多，区别是一个为**执行变量**，一个为**执行命令**，先完成引号里的命令行，然后将其结果替换出来，再重组成新的命令行。

```
echo $(date "+%Y-%m-%d")    #2020-07-01
```

```
echo today is `date "+%Y-%m-%d"`    #today is 2020-07-01
```

2.2.3 数学运算





2.2.3.1 (())

双小括号命令是用来执行**数学表达式**的，可以在其中进行各种逻辑运算、数学运算，也支持更多的运算符（如++、--等）。

```
echo $(((5 * 2)))    #10
```

在(()) 中的**变量名称**，可于其前面加 **\$ 符号**来执行替换，**也可以不用**。

```
i=5  
echo $(((i=$i*2)))    #10  
echo $(((i=i*2)))     #20
```

2.2.3.2 \$(())

\$(())的缩写。

```
echo $((i*2))         #40  
echo $((i*2))         #40
```

2.3 串行与并行

shell脚本默认是**按顺序串行**执行的，使用&可以将一个命令放在后台运行，从而使shell脚本能够继续往后执行：

```
sleep 5 &  
echo "done"
```

上面的脚本执行后会立即打印出"done"，sleep命令被扔给后台执行，不会阻塞脚本执行。

如果想要在进入下个循环前，**必须**等待上个后台命令执行完毕，可以使用wait命令：

```
sleep 5 &
```





```
wait
```

```
echo "done"
```

这样，需要等待5s后才能在屏幕上看到"done"。

3. 脚本实现

3.1 ODS层

3.1.1 建库shell脚本

简单的hive执行sql命令。

```
#!/bin/bash
HIVE_HOME=/usr/bin/hive

${HIVE_HOME} -S -e "....."
```

完整脚本：



create_ods_table.sh

3.1.2 首次shell脚本

简单的sqoop脚本命令，新增同步、更新同步数据时，数据日期DT字段需要用到昨天日期变量。

```
#!/bin/bash

#SQOOP_HOME=/opt/cloudera/parcels/CDH-6.2.1-1.cdh6.2.1.p0.1425774/bin/sqoop
SQOOP_HOME=/usr/bin/sqoop

if [[ $1 == "" ]];then
    TD_DATE=`date -d '1 days ago' "+%Y-%m-%d"`
else
    TD_DATE=$1
fi
```





```
echo '=====开始全量导入====='
```

全量

```
/usr/bin/sqoop import "-Dorg.apache.sqoop.splitter.allow_text_splitter=true" \  
--driver 'com.mysql.jdbc.Driver' \  
--connect \  
'jdbc:mysql://192.168.88.80:3306/yipin?enabledTLSProtocols=TLSv1.2&useUnicode=true&characterE  
ncoding=UTF-8&autoReconnect=true' \  
--username root \  
--password 123456 \  
--query "select * from t_district where l=1 and \${CONDITIONS}" \  
--hcatalog-database yp_ods \  
--hcatalog-table t_district \  
-m 1  
wait
```

新增同步

```
/usr/bin/sqoop import "-Dorg.apache.sqoop.splitter.allow_text_splitter=true" \  
--connect \  
'jdbc:mysql://192.168.88.80:3306/yipin?useUnicode=true&characterEncoding=UTF-8&autoReconnect=  
true' \  
--username root \  
--password 123456 \  
--query "select *, '\${TD_DATE}' as dt from t_goods_evaluation where l=1 \${CONDITIONS}" \  
--hcatalog-database yp_ods \  
--hcatalog-table t_goods_evaluation \  
-m 1  
wait
```

新增和更新同步

```
/usr/bin/sqoop import "-Dorg.apache.sqoop.splitter.allow_text_splitter=true" \  
--connect \  
'jdbc:mysql://192.168.88.80:3306/yipin?useUnicode=true&characterEncoding=UTF-8&autoReconnect=  
true' \  
--username root \  
--password 123456 \  
--query "select *, '\${TD_DATE}' as dt from t_store where l=1 and \${CONDITIONS}" \  
--hcatalog-database yp_ods \  
--hcatalog-table t_store \  
-m 1  
wait
```




```
-m 1
wait

echo '=====success====='
```

完整脚本：



sqoop_import.sh

3.1.3 循环Shell脚本

循环执行时，sqoop的sql查询条件，需要用到昨天日期变量。

```
#!/bin/bash

#SQOOP_HOME=/opt/cloudera/parcels/CDH-6.2.1-1.cdh6.2.1.p0.1425774/bin/sqoop
SQOOP_HOME=/usr/bin/sqoop
if [[ $1 == "" ]];then
    TD_DATE=`date -d '1 days ago' "+%Y-%m-%d"`
else
    TD_DATE=$1
fi

echo '=====开始增量导入===== '

# 全量表
/usr/bin/sqoop import "-Dorg.apache.sqoop.splitter.allow_text_splitter=true" \
--connect
'jdbc:mysql://192.168.88.80:3306/yipin?useUnicode=true&characterEncoding=UTF-8&autoReconnect=
true' \
--username root \
--password 123456 \
--query "select * from t_district where l=1 and \$CONDITIONS" \
--hcatalog-database yp_ods \
--hcatalog-table t_district \
-m 1
wait

.....
```





```
# 新增表

/usr/bin/sqoop import "-Dorg.apache.sqoop.splitter.allow_text_splitter=true" \
--connect
'jdbc:mysql://192.168.88.80:3306/yipin?useUnicode=true&characterEncoding=UTF-8&autoReconnect=
true' \
--username root \
--password 123456 \
--query "select *, '${TD_DATE}' as dt from t_goods_evaluation where l=1 and (create_time between
'${TD_DATE} 00:00:00' and '${TD_DATE} 23:59:59') and \${CONDITIONS}" \
--hcatalog-database yp_ods \
--hcatalog-table t_goods_evaluation \
-m 1
wait

.....

# 新增和更新同步

/usr/bin/sqoop import "-Dorg.apache.sqoop.splitter.allow_text_splitter=true" \
--connect
'jdbc:mysql://192.168.88.80:3306/yipin?useUnicode=true&characterEncoding=UTF-8&autoReconnect=
true' \
--username root \
--password 123456 \
--query "select *, '${TD_DATE}' as dt from t_store where l=1 and ((create_time between '${TD_DATE}
00:00:00' and '${TD_DATE} 23:59:59') or (update_time between '${TD_DATE} 00:00:00' and '${TD_DATE}
23:59:59')) and \${CONDITIONS}" \
--hcatalog-database yp_ods \
--hcatalog-table t_store \
-m 1
wait

.....

echo '=====success====='
```

完整脚本：



sqoop_import_inc.sh





3.2 DWD层

3.2.1 事实表

3.2.1.1 建库shell脚本



create_fact_tables.sh

3.2.1.2 首次shell脚本



insert_fact.sh

3.2.1.3 循环Shell脚本

循环执行时，拉链表的条件需要用到昨天日期变量。

```
#!/bin/bash
HIVE_HOME=/usr/bin/hive

if [[ $1 == "" ]];then
    TD_DATE=`date -d '1 days ago' +%Y-%m-%d`
else
    TD_DATE=$1
fi

${HIVE_HOME} -S -e "
--分区配置
SET hive.exec.dynamic.partition=true;
SET hive.exec.dynamic.partition.mode=nonstrict;
set hive.exec.max.dynamic.partitions.pernode=10000;
set hive.exec.max.dynamic.partitions=100000;
set hive.exec.max.created.files=150000;

=====订单一事实表（拉链表）=====
-- 1、ODS抽取新增/变更数据
-- 略
-- 2、重建临时表
```



```
DROP TABLE if EXISTS yp_dwd.fact_shop_order_tmp;
CREATE TABLE yp_dwd.fact_shop_order_tmp(
    id string COMMENT '根据一定规则生成的订单编号',
    order_num string COMMENT '订单序号',
    buyer_id string COMMENT '买家的userId',
    store_id string COMMENT '店铺id',
    order_from string COMMENT '此字段可以转换 1.安卓\; 2.ios\; 3.小程序H5 \; 4.PC',
    order_state int COMMENT '订单状态:1.已下单\; 2.已付款, 3. 已确认 \;4.配送\; 5.已完成\; 6.退款\;7.已取消',
    create_date string COMMENT '下单时间',
    finnshed_time timestamp COMMENT '订单完成时间,当配送员点击确认送达时,进行更新订单完成时间,后期需要根据订单完成时间,进行自动收货以及自动评价',
    is_settlement tinyint COMMENT '是否结算\;0.待结算订单\; 1.已结算订单\;',
    is_delete tinyint COMMENT '订单评价的状态:0.未删除\; 1.已删除\;(默认0)',
    evaluation_state tinyint COMMENT '订单评价的状态:0.未评价\; 1.已评价\;(默认0)',
    way string COMMENT '取货方式:SELF自提\;SHOP店铺负责配送',
    is_stock_up int COMMENT '是否需要备货 0: 不需要    1: 需要    2:平台确认备货 3:已完成备货 4平台已经将货物送至店铺 ',
    create_user string,
    create_time string,
    update_user string,
    update_time string,
    is_valid tinyint COMMENT '是否有效 0: false\; 1: true\;  订单是否有效的标志',
    end_date string COMMENT '拉链结束日期')
COMMENT '订单表'
partitioned by (start_date string)
row format delimited fields terminated by '\t'
stored as orc
tblproperties (('orc.compress' = 'SNAPPY'));
-- 3、合并新旧数据
insert overwrite table yp_dwd.fact_shop_order_tmp partition (start_date)
select * from
(
-- 一、ods表的新数据
select
    id,
    order_num,
    buyer_id,
    store_id,
    case order_from
```





```
when 1
then 'android'
when 2
then 'ios'
when 3
then 'miniapp'
when 4
then 'pcweb'
else 'other'
end
as order_from,
order_state,
create_date,
finnshed_time,
is_settlement,
is_delete,
evaluation_state,
way,
is_stock_up,
create_user,
create_time,
update_user,
update_time,
is_valid,
'9999-99-99' end_date,
'${TD_DATE}' as start_date
from yp_ods.t_shop_order where dt='${TD_DATE}'

union all
```

-- 二、历史拉链表数据，并根据up_id判断更新end_time有效期

```
select
fso.id,
fso.order_num,
fso.buyer_id,
fso.store_id,
fso.order_from,
fso.order_state,
fso.create_date,
fso.finnshed_time,
fso.is_settlement,
fso.is_delete,
```





```
fso.evaluation_state,
fso.way,
fso.is_stock_up,
fso.create_user,
fso.create_time,
fso.update_user,
fso.update_time,
fso.is_valid,
--3、更新end_time: 如果没有匹配到变更数据，或者当前已经是无效的历史数据，则保留原始end_time过期时间；
否则变更end_time时间为前天（昨天之前有效）
    if (up.id is null or fso.end_date < '9999-99-99', fso.end_date, date_add('${TD_DATE}', -1))
end_time,
    fso.start_date
from yp_dwd.fact_shop_order fso left join
(
    select
    *
    from yp_ods.t_shop_order
    where dt = '${TD_DATE}'
) up
on fso.id = up.id
--4、时间限制：如果订单的变更周期是30天则可加上此条件，结果会按照所属分区进行覆盖插入
--where fso.start_date >= date_add('${TD_DATE}', -30)
) his
order by his.id, start_date;

-- 4、临时表覆盖拉链表
INSERT OVERWRITE TABLE yp_dwd.fact_shop_order partition (start_date)
SELECT * from yp_dwd.fact_shop_order_tmp;
```

完整脚本：



insert_fact_append.sh

3.2.2 维度表



3.2.2.1 建库shell脚本



create_dim_tables.sh

3.2.2.2 首次shell脚本



insert_dim.sh

3.2.2.3 循环Shell脚本



insert_dim_append.sh

3.3 DWB层

3.3.1 建库shell脚本



create_dwb_table.sh

3.3.2 首次shell脚本



insert_dwb.sh

3.3.3 循环Shell脚本

DWB关联查询时，需要将30天内的数据（上个月1日至今）覆盖插入，用到了变量：上个月1日。

```
#!/bin/bash  
HIVE_HOME=/usr/bin/hive
```





#上个月1日

```
Last_Month_DATE=$(date -d "-1 month" +%Y-%m-01)
```

```
${HIVE_HOME} -S -e "
```

```
-----订单宽表-----
```

```
--增量插入
```

```
insert overwrite table yp_dwb.dwb_order_detail partition (dt)
```

```
select
```

```
.....
```

```
--订单表
```

```
from yp_dwd.fact_shop_order o
```

```
--订单副表
```

```
left join yp_dwd.fact_shop_order_address_detail od on o.id = od.id and od.end_date='9999-99-99'
```

```
--订单组表
```

```
left join yp_dwd.fact_shop_order_group og on o.id = og.order_id and og.end_date='9999-99-99'
```

```
--订单组支付表
```

```
left join yp_dwd.fact_order_pay op on og.group_id = op.group_id
```

```
--退款单表
```

```
left join yp_dwd.fact_refund_order refund on refund.order_id=o.id and  
refund.end_date='9999-99-99'
```

```
--结算单表
```

```
left join yp_dwd.fact_order_settle os on os.order_id=o.id and os.end_date='9999-99-99'
```

```
--订单商品快照
```

```
left join yp_dwd.fact_shop_order_goods_details ogoods on ogoods.order_id=o.id and  
ogoods.end_date='9999-99-99'
```

```
--订单评价表
```

```
left join yp_dwd.fact_goods_evaluation e on e.order_id=o.id and e.is_valid=1
```

```
--订单配送表
```

```
left join yp_dwd.fact_order_delievery_item d on d.shop_order_id=o.id and  
d.dispatcher_order_type=1 and d.is_valid=1 and d.end_date='9999-99-99'
```

```
where o.end_date='9999-99-99'
```

```
-- 读取上个月1日至今的数据
```

```
and SUBSTRING(o.create_date,1,10) >= '${Last_Month_DATE}' and o.start_date >=  
'${Last_Month_DATE}'
```

```
;
```

完整脚本:



insert_dwb_append.sh





3.4 DWS层

3.4.1 建库shell脚本



create_dws.sh

3.4.2 首次shell脚本



insert_dws.sh

3.4.3 循环Shell脚本

```
#!/bin/bash
HIVE_HOME=/usr/bin/hive

#昨天
if [[ $1 == "" ]];then
    TD_DATE=`date -d '1 days ago' +%Y-%m-%d`
else
    TD_DATE=$1
fi

${HIVE_HOME} -S -e "
-----销售主题-----
insert into yp_dws.dws_sale_daycount
with order_base as
(
    select
        ....,
        row_number() over(partition by o.order_id) rn,
        ....
    from yp_dwb.dwb_order_detail o
```





```
left join yp_dwb.dwb_shop_detail s on o.store_id=s.id
left join yp_dwb.dwb_goods_detail g on o.goods_id=g.id
where o.is_pay=1
-- 计算昨天的数据
and o.dt='${TD_DATE}'
)
select
.....
-- 销售收入
,case
    when grouping(brand_id)=0
    then sum(if(brand_goods_rn=1, coalesce(goods_price, 0), 0))
    when grouping(min_class_id)=0
    then sum(if(min_class_goods_rn=1, coalesce(goods_price, 0), 0))
    when grouping(mid_class_id)=0
    then sum(if(mid_class_goods_rn=1, coalesce(goods_price, 0), 0))
    when grouping(max_class_id)=0
    then sum(if(max_class_goods_rn=1, coalesce(goods_price, 0), 0))
    when grouping(store_id)=0
    then sum(if(store_rn=1, coalesce(order_amount, 0), 0))
    when grouping(trade_area_id)=0
    then sum(if(trade_area_rn=1, coalesce(order_amount, 0), 0))
    when grouping(city_id)=0
    then sum(if(city_rn=1, coalesce(order_amount, 0), 0))
    when grouping(dt)=0
    then sum(if(rn=1, coalesce(order_amount, 0), 0))
    else null
end
as sale_amt,
-- .....
from order_base o
group by
grouping sets (
    dt,
    (dt, city_id, city_name),
    (dt, trade_area_id, trade_area_name, city_id, city_name),
    (dt, store_id, store_name, trade_area_id, trade_area_name, city_id, city_name),
    (dt, brand_id, brand_name),
    (dt, max_class_id, max_class_name),
    (dt, mid_class_id, mid_class_name, max_class_id, max_class_name),
    (dt, min_class_id, min_class_name, mid_class_id, mid_class_name, max_class_id,
max_class_name)
```





);



insert_dws_append.sh

3.5 DM层

3.5.1 建库shell脚本



create_dm.sh

3.5.2 首次shell脚本



insert_dm.sh

3.5.3 循环Shell脚本

销售主题，要重新计算年月日数据，条件需要用到昨天所在年的起始日期：\${TD_DATE_YEAR}；

商品主题和用户主题，需要进行累加，获取昨日数据和上个月数据时，都需要\${TD_DATE}变量。

```
#!/bin/bash
HIVE_HOME=/usr/bin/hive

#昨天
if [[ $1 == "" ]];then
    TD_DATE=`date -d '1 days ago' +%Y-%m-%d`
else
    TD_DATE=$1
fi

TD_DATE_YEAR=`date --date="$TD_DATE" +%Y`

${HIVE_HOME} -S -e "
```





-----销售主题-----

insert into yp_dm.dm_sale

-- 获取日期数据 (周、月的环比/同比日期)

with dt1 as (

select

dim_date_id, date_code

, date_id_mom -- 与本月环比的上月日期

, date_id_mym -- 与本月同比的上年日期

, year_code

, month_code

, year_month --年月

, day_month_num --几号

, week_day_code --周几

, year_week_name_cn --年周

from yp_dwd.dim_date

),

groupby as (

select

.....

CASE WHEN grouping(dc.store_id, dc.trade_area_id, dc.city_id, dc.brand_id,
dc.min_class_id, dc.mid_class_id, dc.max_class_id)=15 and group_type='store'

THEN sum(dc.sale_amt)

WHEN grouping(dc.store_id, dc.trade_area_id, dc.city_id, dc.brand_id,
dc.min_class_id, dc.mid_class_id, dc.max_class_id)=79 and group_type='trade_area'

THEN sum(dc.sale_amt)

WHEN grouping(dc.store_id, dc.trade_area_id, dc.city_id, dc.brand_id,
dc.min_class_id, dc.mid_class_id, dc.max_class_id)=111 and group_type='city'

THEN sum(dc.sale_amt)

WHEN grouping(dc.store_id, dc.trade_area_id, dc.city_id, dc.brand_id,
dc.min_class_id, dc.mid_class_id, dc.max_class_id)=119 and group_type='brand'

THEN sum(dc.sale_amt)

WHEN grouping(dc.store_id, dc.trade_area_id, dc.city_id, dc.brand_id,
dc.min_class_id, dc.mid_class_id, dc.max_class_id)=120 and group_type='min_class'

THEN sum(dc.sale_amt)

WHEN grouping(dc.store_id, dc.trade_area_id, dc.city_id, dc.brand_id,
dc.min_class_id, dc.mid_class_id, dc.max_class_id)=124 and group_type='mid_class'

THEN sum(dc.sale_amt)

WHEN grouping(dc.store_id, dc.trade_area_id, dc.city_id, dc.brand_id,
dc.min_class_id, dc.mid_class_id, dc.max_class_id)=126 and group_type='max_class'

THEN sum(dc.sale_amt)

when grouping(dc.store_id, dc.trade_area_id, dc.city_id, dc.brand_id,
dc.min_class_id, dc.mid_class_id, dc.max_class_id)=127 and group_type='all'

then sum(dc.sale_amt)





```
ELSE null
end
as sale_amt,
.....

from yp_dws.dws_sale_daycount dc
left join dtl on dc.dt = dtl.date_code
WHERE dc.dt >= '${TD_DATE_YEAR}-01-01'
group by
grouping sets (
-- 年，注意养成加小括号的习惯
(dtl.year_code, group_type),
(dtl.year_code, city_id, city_name, group_type),
(dtl.year_code, city_id, city_name, trade_area_id, trade_area_name, group_type),
(dtl.year_code, city_id, city_name, trade_area_id, trade_area_name, store_id, store_name,
group_type),
(dtl.year_code, brand_id, brand_name, group_type),
(dtl.year_code, max_class_id, max_class_name, group_type),
(dtl.year_code, max_class_id, max_class_name, mid_class_id, mid_class_name, group_type),
(dtl.year_code, max_class_id, max_class_name, mid_class_id, mid_class_name, min_class_id,
min_class_name, group_type),
-- 月
.....
-- order by time_type desc
)
select
.....
from groupby
where sale_amt is not null
;

-----商品主题宽表-----
--1、重建临时表
.....

--2、合并新旧数据
-- 增量
insert into yp_dm.dm_sku_tmp
-- dm旧数据
with old as (
```





```
select
    sku_id, sku_name,
    order_count,
    .....
from yp_dm.dm_sku
),
-- 昨天 的dws新数据
last_day as (
    select
        sku_id,
        max(sku_name) sku_name,
        sum(coalesce(order_count,0)) as order_count,
        .....
    from yp_dws.dws_sku_daycount
    where dt='${TD_DATE}'
    group by sku_id
),
-- 总累积数值=旧数据+昨日新数据
all as (
    select
        'all' time_type,
        null year_code,
        null year_month,
        coalesce(last_day.sku_id,old.sku_id) sku_id,
        coalesce(last_day.sku_name,old.sku_name) sku_name,
        -- 订单 累积历史数据
        coalesce(old.order_count,0) + coalesce(last_day.order_count,0) order_count,
        coalesce(old.order_num,0) + coalesce(last_day.order_num,0) order_num,
        coalesce(old.order_amount,0) + coalesce(last_day.order_amount,0) order_amount,
        .....
    from old
    full join last_day on old.sku_id=last_day.sku_id
),
-- 月统计值
last_month as (
    select
        'month' time_type,
        substring('${TD_DATE}', 1, 4) year_code,
        substring('${TD_DATE}', 1, 7) year_month,
        sku_id, max(sku_name),
        sum(coalesce(order_count,0)) as order_count,
```



```
.....

from yp_dws.dws_sku_daycount
where dt>=concat(substring('${TD_DATE}', 1, 7), '-01')
group by sku_id
)

-- 合并总累积和月度数据

select * from all
union all
select *
from last_month;

--3、临时表覆盖宽表
delete from yp_dm.dm_sku;
insert into yp_dm.dm_sku
select * from yp_dm.dm_sku_tmp;
```



insert_dm_append.sh

3.6 RPT层

3.6.1 建库shell脚本



create_rpt.sh

3.6.2 首次shell脚本



insert_rpt.sh

3.6.3 循环Shell脚本

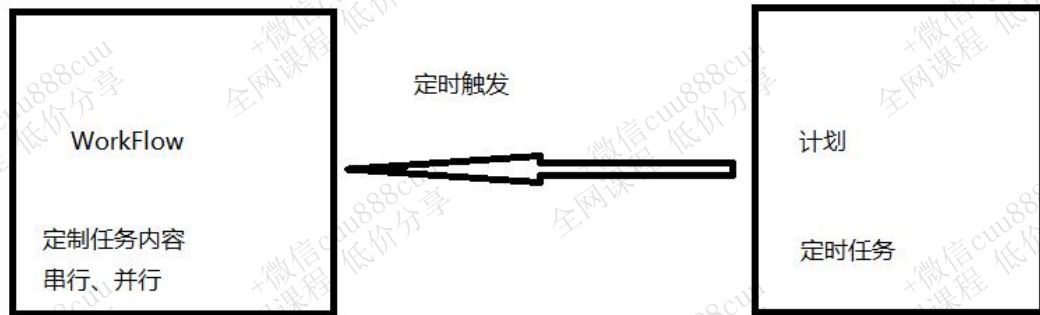


insert_rpt_append.sh





4. 调度



4.1 时区设置

1. ClouderaManager中，hue 配置修改time_zone的值为：Asia/Shanghai

时区

time_zone

Hue (服务范围)

Asia/Shanghai

2. ClouderaManager中，oozie配置修改oozie-site.xml，添加配置项：

name: oozie.processing.timezone

Value: GMT+0800

Description: oozie时区设置为东八区区时

oozie-site.xml 的 Oozie Server 高级配置代码段
(安全阀)

Oozie Server Default Group

以 XML 格式查看

名称

oozie.processing.timezone

值

GMT+0800

说明

oozie时区设置为东八区区时

最终



4.2 创建workflow

4.2.1 上传shell脚本到HDFS:

文件浏览器

搜索文件名

操作

移至垃圾桶

上传

新建

主页 / user / hue / oozie / yipin 垃圾桶

名称	大小	用户	组	权限	日期
↑		hue	hue	drwxrwxrwx	August 29, 2021 01:00 PM
.		hue	hue	drwxr-xr-x	August 29, 2021 02:51 PM
create_dim_tables.sh	11.5 KB	hue	hue	-rw-r--r--	August 29, 2021 01:00 PM
create_dm.sh	5.4 KB	hue	hue	-rw-r--r--	August 29, 2021 01:00 PM
create_dwb_table.sh	8.8 KB	hue	hue	-rw-r--r--	August 29, 2021 02:23 PM
create_dws.sh	4.4 KB	hue	hue	-rw-r--r--	August 29, 2021 01:00 PM
create_fact_tables.sh	16.9 KB	hue	hue	-rw-r--r--	August 29, 2021 01:00 PM
create_ods_table.sh	29.2 KB	hue	hue	-rw-r--r--	August 29, 2021 01:00 PM
create_rpt.sh	6.0 KB	hue	hue	-rw-r--r--	August 29, 2021 02:51 PM
insert_dim.sh	3.3 KB	hue	hue	-rw-r--r--	August 29, 2021 01:01 PM
insert_dim_append.sh	7.6 KB	hue	hue	-rw-r--r--	August 29, 2021 01:01 PM
insert_dm.sh	38.8 KB	hue	hue	-rw-r--r--	August 29, 2021 01:01 PM
insert_dm_append.sh	43.3 KB	hue	hue	-rw-r--r--	August 29, 2021 01:01 PM
insert_dwb.sh	5.4 KB	hue	hue	-rw-r--r--	August 29, 2021 01:01 PM

4.2.2 设置workflow

Oozie 编辑器

操作

yipin

Add a description...

Shell

/user/hue/oozie/yipin/create_ods

参数

文件

Shell

/user/hue/oozie/yipin/create_dim

参数

文件





4.2.3 测试workflow

The screenshot displays the Oozie workflow console interface. On the left, a sidebar shows job details: ID 00000003-2106151015, Name 01875-oozie-oozi-W, User hue, and a progress bar at 88%. The main area shows a sequence of tasks: a start node, followed by Shell tasks for create_ods_table.sh, create_dim_tables.sh, create_fact_tables.sh, and create_dwb_table.sh, each with a status icon. At the top right, there are control buttons: 恢复 (Resume), 暂停 (Pause), 重新运行 (Restart), and 停止 (Stop).

4.3 创建调度计划

4.3.1 创建计划

注意时区要设置正确。



要计划哪个 Workflow?

visit_consult_import

频率?

每 日 在 2 : 51

[隐藏](#)

☐ 高级语法

时区 Asia/Shanghai

自 2020-07-15 20:25

至 9999-07-22 20:25

参数

[+ 添加参数](#)

Save

4.3.2 保存，并提交



4.3.3 查看提交列表



作业浏览器

作业

Workflow

计划

Bundles

SLAs

按 ID、名称、用户筛选...

成功

正在运行

失败

▶ 恢复

|| 暂停

✕ 停止

正在运行

☐	名称	用户	类型	状态	进度	组	已修改	持续时间	Id
☐	plan_visit_consult_import	hue	schedule	RUNNING	100%		2020年7月16日晚上8点39分	1d, 20h, 0s	0000001-200715195252127-gozle-qozi-C

Completed

☐	名称	用户	类型	状态	进度	组	已修改	持续时间	Id
--------------------------	----	----	----	----	----	---	-----	------	----

正在显示 1 至 1 的 1

4.3.4 查看触发记录

作业浏览器

作业

Workflow

计划

Bundles

SLAs

按 ID、名称、用户筛选...

成功

正在运行

失败

在过去

7

天

正在运行

☐	名称	用户	类型	状态	进度	组	开始	持续时间	Id
☐	oozie:action:T=shell;W=visit_consult_import;A=shell-9a56;ID=00000002-200715195252127-oozie-oozi-W	yarn	MAPREDUCE	RUNNING	9.5%	root.users.yarn	2020年7月15日晚上8点39分	35.70s	application_1594625530549_0086
☐	oozie:launcher:T=shell;W=visit_consult_import;A=shell-9a56;ID=00000002-200715195252127-oozie-oozi-W	hue	Oozie Launcher	RUNNING	50%	root.users.hue	2020年7月15日晚上8点39分	1m, 0s	application_1594625530549_0085

Completed

☐	名称	用户	类型	状态	进度	组	开始	持续时间	Id
☐	oozie:action:T=shell;W=visit_consult_import;A=shell-9a56;ID=00000002-200715195252127-oozie-oozi-W	yarn	MAPREDUCE	SUCCEEDED	100%	root.users.yarn	2020年7月15日晚上8点08分	8m, 59s	application_1594625530549_0084
☐	oozie:action:T=shell;W=visit_consult_import;A=shell-9a56;ID=00000002-200715195252127-oozie-oozi-W	yarn	MAPREDUCE	SUCCEEDED	100%	root.users.yarn	2020年7月15日晚上7点59分	8m, 47s	application_1594625530549_0083
☒	oozie:launcher:T=shell;W=visit_consult_import;A=shell-9a56;ID=00000002-200715195252127-oozie-oozi-W	hue	Oozie Launcher	FAILED	100%	root.users.hue	2020年7月15日晚上7点58分	18m, 50s	application_1594625530549_0082
☐	QueryResult.jar	root	MAPREDUCE	SUCCEEDED	100%	root.users.root	2020年7月15日	3m, 58s	application_1594625530549_0081



作业浏览器 作业 Workflow 计划 Bundles SLAs

0000002-200715195252127-oozie-oozi-W > oozie:launcher:T=shell:W=visit_consult_import:A=shell-9a56:ID=0000002-200715195252127-oozie-oozi-W

ID
application_15946255
30549_0085

日志 尝试

类型
Oozie Launcher

进度
50%

状态
RUNNING

开始时间
2020年7月15日晚上...

完成时间
2020年7月15日晚上...

持续时间
2m, 38s

stdout

stderr

```
20/07/15 20:40:29 INFO mapreduce.Job: map 11% reduce 0%
20/07/15 20:40:35 INFO mapreduce.Job: map 12% reduce 0%
20/07/15 20:40:40 INFO mapreduce.Job: map 13% reduce 0%
20/07/15 20:40:44 INFO mapreduce.Job: map 14% reduce 0%
20/07/15 20:40:49 INFO mapreduce.Job: map 15% reduce 0%
20/07/15 20:40:54 INFO mapreduce.Job: map 16% reduce 0%
20/07/15 20:40:59 INFO mapreduce.Job: map 17% reduce 0%
20/07/15 20:41:04 INFO mapreduce.Job: map 18% reduce 0%
20/07/15 20:41:09 INFO mapreduce.Job: map 19% reduce 0%
20/07/15 20:41:15 INFO mapreduce.Job: map 20% reduce 0%
20/07/15 20:41:21 INFO mapreduce.Job: map 21% reduce 0%
20/07/15 20:41:28 INFO mapreduce.Job: map 22% reduce 0%
20/07/15 20:41:33 INFO mapreduce.Job: map 23% reduce 0%
20/07/15 20:41:38 INFO mapreduce.Job: map 24% reduce 0%
```

4.3.5 执行成功

主页 / user / hive / warehouse / itcast_ods.db / web_chat_ems_ods

☐

名称

大小

用户

☐

↑

hue

☐

↓

hue

☐

↑

starts_time=2019-07-01

root

☐

↓

starts_time=2019-07-25

yarn

显示 45 的 2 项目

